

XRPL Validator — Daily Engineering Report • 2026-09-08

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **11 commits** (xrpl-validator 11)
- **11 engine findings fixed** (F227–F237)
- Amendment **fixCleanup3_3_0** held majority (since 2026-08-28 11:15 UTC); activation due ~2026-09-11 11:15 UTC

Engine findings fixed

- **fix(xrpl-ledger): F227 — a self-payment keeps its lined terminal ripple step** (d2344e4 , xrpl-validator) When the destination is itself the deliver issuer, toStrand appends no issuer hop (PaySteps.cpp:308-313) and the strand ends with DirectStepI(lastIssuer → dst): a plain trust-line step that rippled keeps whenever the line exists (DirectStepI::check refuses only a missing one), delivering the last hop's issue into the destination's line with it. Our builder dropped every terminal same-currency issuer change by shape ...
- **fix(xrpl-ledger): F228 — a payment's tail turn anchors the pool on the raw book tip** (54d1057 , xrpl-validator) BookStep::forEachOffer runs tryAMM(offers.tip().quality()) before it looks at the tip itself, and a PAYMENT step's qualityThreshold(lob) is the lob: the pool's offer is anchored on the raw tip even when that tip sits beyond limitQuality and is never crossed. The threshold → nullopt override that yields the unanchored max offer belongs to BookOfferCrossingStep alone (BookStep.cpp:476-481). Our tail turn applied the ...
- **fix(xrpl-ledger): F229 — NFTokenPage keys are the owner's account bytes, not a hash** (5e46961 , xrpl-validator) rippled's keylet::nftokenPageMin(owner) is the owner's twenty account bytes followed by twelve zero bytes and nftokenPageMax the same with 0xFF×12 (Indexes.cpp:418-431); a token's page is the base with the token's low 96 bits, and the owner's last page always sits at the max key. Our nftoken_page_key hashed 0x0050 || account the way the indexHash keylets do, so finding 222's NFT-holder check on AccountDelete ...
- **fix(xrpl-ledger): F230 — the stream reaps an expired offer before the self-cross check, and steps on after a consumed fill** (ca724f9 , xrpl-validator) OfferStream::step removes an EXPIRED offer before any self-cross or quality logic ever sees it (OfferStream.cpp:192, "Removing expired offer"), whoever owns it and wherever it sits against the limit. Our shared stepper tested a taker-owned offer against the limit first — right for a LIVE self-offer, which ends the pass — and so caught the taker's own expired offer and left it standing. And the bridged crossing's ...
- **fix(xrpl-ledger): F231 — an in-limited fill's output is clamped to the offer's own** (e10f8d2 , xrpl-validator) Quality::ceilIn derives the output from the limited input at the offer's stored rate and then clamps it to the offer's own output (Quality.cpp ceilInImpl: "if (result.out > amount.out) result.out = amount.out"). A rate rounded to sixteen digits can otherwise hand out more than the offer holds, and our walk re-derived the maker's give from the clamped input without that bound.

- **fix(xrpl-ledger): F232 — a pool turn that ends the iteration leaves the tip in place; judge the beyond-strict sweep there, post-turn** (`f7ef2ee` , `xrpl-validator`) `rippled's BookStep::forEachOffer` consults the AMM once per iteration, at the iteration's first tip, and `execOffer` ends the iteration on the first offer whose quality differs from the anchor (`*ofrQ != offer.quality()`). So a pool turn whose slice quality sits off the level's own quality ends the iteration BEFORE that level's offers are reached, and the NEXT iteration opens with its tip on the very same level — where ...
- **fix(xrpl-ledger): F233 — bridge peeks reap dead offers only on the strands rippled flows this round** (`80eaadd` , `xrpl-validator`) `rippled` removes the expired and unfunded offers a stream steps past only inside a strand it FLOWS, and in an offer crossing a strand is flowed only when its quality upper bound — raw book tips (`BookTip::step` reads the first directory entry as it stands, dead or not) and pool spots — sits inside the limit (`StrandFlow.h qualityUpperBound < limitQuality => continue`). A fill taken on the direct strand says nothing ...
- **fix(xrpl-ledger): F234 — CheckCash cashes through the payment engine: SendMax buys what it can, DeliverMin is a floor, XRP shortfall is tecUNFUNDED_PAYMENT** (`d959488` , `xrpl-validator`) `rippled` cashes a check with the payment engine (`CheckCash.cpp:331-577`), not with a transfer of the requested amount: XRP `xrpDeliver = DeliverMin ? max(DeliverMin, min(sendMax, srcLiquid)) : Amount` with `srcLiquid` the writer's liquid XRP after its reserve for the check is released; a shortfall is `tecUNFUNDED_PAYMENT`.
- **fix(xrpl-ledger): F235 — NFTokenCreateOffer's preclaim is rippled's tokenOfferCreatePreclaim: an unfunded buyer is tecUNFUNDED_OFFER** (`c6751ea` , `xrpl-validator`) Our `NFTokenCreateOffer` preclaim checked the token's existence, the Expiration and the Destination/Owner disallow-incoming flags, and nothing else. `rippled's nft::tokenOfferCreatePreclaim` (`NFTokenHelpers.cpp:811-882`) adds, in this order: a transfer-fee NFT priced in an IOU needs its issuer alive, holding a line for that IOU unless it issues it, and not frozen on it (`tecNO_ISSUER / tecNO_LINE / tecFROZEN`); the account ...
- **fix(xrpl-ledger): F236 — the direct walk rests an offer's in side as TakerGets less the gross fold divided once by the gateway rate** (`1ad42e1` , `xrpl-validator`) `OfferCreate.cpp:490-517` rests what remains of a crossed offer's in side as `afterCross.in = takerGets - divideRound(result.actualAmountIn, gatewayXferRate, roundUp = true)`, where `actualAmountIn` is the sorted-multiset fold of the iterations' gross ins (Finding 226): one division of the total, rounded up. The bridged walk already rested it this way; the direct walk carried its running remainder — `TakerGets` less each ...
- **fix(xrpl-ledger): F237 — a bridged round takes every leg-A offer at the tip's quality into one leg-B slice** (`ab1a5c4` , `xrpl-validator`) `rippled's BookStep::forEachOffer` consumes, in ONE pass, every live offer at the tip's quality (`execOffer` keeps going while `ofrQ` matches, up to the strand's limits), and the next step then prices their combined XRP once — for a pool on leg B, one slice for the whole level. Leg B of the bridged round has grouped its level this way since Finding 137; leg A still took one offer per round, so a level of several ...

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_3_0	2026-08-28 11:15 UTC	2026-09-11 11:15 UTC

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Fund NAV Oracle & Queued MPT Redemption** (rwa) Add an `ltFUNDNAV` ledger object (issuer owner-reserved, `keylet::fundNav(mptIssuanceID)`) holding `sfNAVValue`, `sfNAVTimestamp` and `sfNAVOracleID` referencin
- **Policy-Bound Recurring Pull Authorizations** (payments) Extend `g18's AccountPolicy` object with a bounded `STArray sfRecurringAuthorizations` (max 32 entries), each holding `sfAuthorizedPayee`, `sfMaxAmountPerPer`
- **Rotating Viewing Keys for Confidential MPT** (privacy) Add `sfMPTViewingKey` and `sfMPTViewingKeyVersion` `SFields` to `MPTokenIssuance` for issuers using `ConfidentialTransfer`-flagged MPT, plus a new `MPTRotateView`

Appendix — commits

Time	Repo	Commit	Subject
05:33	xrpl-validator	d2344e4	fix(xrpl-ledger): F227 — a self-payment keeps its lined terminal ripple step
05:43	xrpl-validator	54d1057	fix(xrpl-ledger): F228 — a payment's tail turn anchors the pool on the raw book tip
08:32	xrpl-validator	5e46961	fix(xrpl-ledger): F229 — <code>NFTokenPage</code> keys are the owner's account bytes, not a hash
08:45	xrpl-validator	ca724f9	fix(xrpl-ledger): F230 — the stream reaps an expired offer before the self-cross check, and steps on after a consumed fill
11:08	xrpl-validator	e10f8d2	fix(xrpl-ledger): F231 — an in-limited fill's output is clamped to the offer's own
12:26	xrpl-validator	f7ef2ee	fix(xrpl-ledger): F232 — a pool turn that ends the iteration leaves the tip in place; judge the beyond-strict sweep there, post-turn
12:47	xrpl-validator	80eaadd	fix(xrpl-ledger): F233 — bridge peeks reap dead offers only on the strands rippled flows this round
13:04	xrpl-validator	d959488	fix(xrpl-ledger): F234 — <code>CheckCash</code> cashes through the payment engine: <code>SendMax</code> buys what it can, <code>DeliverMin</code> is a floor, <code>XRP shortfall</code> is <code>tecUNFUNDED_PAYMENT</code>
14:16	xrpl-validator	c6751ea	fix(xrpl-ledger): F235 — <code>NFTokenCreateOffer's</code> preclaim is rippled's <code>tokenOfferCreatePreclaim</code> : an unfunded buyer is <code>tecUNFUNDED_OFFER</code>
18:54	xrpl-validator	1ad42e1	fix(xrpl-ledger): F236 — the direct walk rests an offer's in side as <code>TakerGets</code> less the gross fold divided once by the gateway rate
22:24	xrpl-validator	ab1a5c4	fix(xrpl-ledger): F237 — a bridged round takes every leg-A offer at the tip's quality into one leg-B slice